

03 — Reconstruct, build and audit a three-box model

Learning goals: reconstruct a scientific flux diagram; translate it through Excel records into native ESBMTK objects; explain contrasting organic/carbonate effects on pCO₂; check the connections, whole-system budgets and a saved starting state.

Provisional time: 55 minutes. Diagram, paired flux equations and Excel reconciliation (15), four native mappings (20), boundary budget and checks (15), explanation (5).

Follow **reconstruct** → **reconcile** → **map** → **run** → **check** → **explain**. Your diagram and paired flux equations are the specification for your code and for 04. Repeated loops, chemistry/sediment wiring, integration and plotting are supplied. [Teaching goals and instructor timing notes](#).

Reading key: **Key term** = concept to notice. Blue **Question** panels identify student work; purple **Instructor answer** panels appear only in the instructor sheet. Code labels distinguish **Choose and explain**, **Understand and run**, and **Supplied implementation**. This practical is ungraded. Keep your explanations here; no separate submission is required. The [coding cheatsheet](#) is optional lookup support; essential syntax is explained locally.

LECTURER REVIEW | INSTRUCTOR EDITION

Executed in ESBMTK314. All instructor solutions, code, execution outputs and optional disclosures are included. No Jupyter installation is needed to read this PDF.

Source: 03_boudreau_three_box_model.ipynb

Execution: 2026-09-24. The source notebooks are unchanged.

A1. What changes from 02?

This is a Boudreau-like benchmark with a different specification. Do not carry over 02's fitted k , ratio-derived geometry or total carbon inventory.

Feature	02	03/04
Geometry and conditions	Two layers, uniform T/S/P	Low-/high-latitude surface and deep boxes; box-specific conditions
Organic export	$kDIC_s(t)$	Fixed POC export
Carbonate processes	Absent	PIC export, dissolution and net burial
Active atm–ocn boundary	Closed except the pulse	Weathering and net burial; forcing added in 04
Reference state	Calibrated constraints	Saved values from a long benchmark run

A **model state** is the set of current values that the model evolves: here, ocn DIC/TA, atm CO₂ and the **snowline** (the deepest boundary of existing reactive carbonate sediment), along with auxiliary chemistry values. A **restart** uses saved values from an earlier run as the starting state of a new run. In C3 we will load a supplied state from a long benchmark run, then check that your reconstructed model changes little over 20 years. This avoids repeating the long calculation in class.

POC and PIC mean **particulate organic and inorganic carbon**. Here POC export includes full deep remineralization: conversion back to dissolved carbon. There is no explicit particle or nutrient inventory. The **rain ratio** is PIC/POC carbon export; the workbook prescribes this ratio and POC, then the loader derives PIC.

Read [ESBMTK section 3 and Figure 3](#) (printed p. 1162; [online version](#)). Use these corrections:

- The organic-export label in the prose should be **F5**, not F3; F3 denotes circulation.

- The benchmark uses **F6/F5 = PIC/POC = 0.3**, not the inverted ratio in the prose.
- Weathering supplies **dissolved inorganic carbon** here, not organic carbon.

Figure 3 is an incomplete specification: its arrows omit equations, stoichiometry and an explicit dissolution return. Reconstruct those properties using the following assumptions and workbook inputs.

Process rules for your equations

- Water carries both dissolved tracers at the **source concentration**. Draw the circulation loop $L_b \rightarrow H_b \rightarrow D_b \rightarrow L_b$ and both $H_b \leftrightarrow D_b$ mixing directions.
- Each surface exchanges CO₂ with the same finite atm. Use invasion and outgassing as in 01/02.
- Only L_b exports POC/PIC. The table supplies the benchmark's biological, carbonate and weathering assumptions beside their rows.
- Draw a distinct carbonate/sediment **process module** with an explicit dissolution return. Its snowline records sediment history; the module has no explicit sediment-carbon inventory. Detailed sediment equations remain [optional](#).

```
# Supplied implementation: run this support code as provided.
from pathlib import Path
import sys
import pandas as pd
import numpy as np

ROOT = next(p for p in (Path.cwd().resolve(), *Path.cwd().resolve().parents)
             if (p / 'teaching_config.py').is_file())
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))

from esbmtk import (
    Q_, ConnectionProperties, GasReservoir, Model, Species2Species,
    add_carbonate_system_1, add_carbonate_system_2,
    create_bulk_connections, initialize_reservoirs,
)
from model import postprocess_carbonate_horizons, run_model, standard_diagnostics
from presets import load_boudreau_parameters
from reservoir_inputs import reservoir_inventory_rows
from model_inputs import read_model_tables

WORKBOOK = ROOT / 'data' / 'Boudreau_2010' / 'model_definition.xlsx'
P = load_boudreau_parameters(WORKBOOK)
input_tables = read_model_tables(WORKBOOK)
STATE = ROOT / 'data' / 'Boudreau_2010' / 'steady_state'
from teaching_specification import BASELINE_PARAMETERS, workbook_connections
from teaching_audits import audit_benchmark_graph, audit_complete_model, audit_restart_drift
```

A2. Inspect the numerical inputs

The workbook owns areas, volumes, box-specific T/S/P and initial states. The atmosphere has its own mole-fraction and total-air units. **The workbook concentrations initialize construction; C3 replaces them with the saved starting values.** Geometry, thermodynamic settings and process parameters stay the same.

For now inspect only reservoir and baseline parameter tables. Connection rows come after your reconstruction. Excel editing is not required.

```
# Understand and run: read these supplied inputs before drawing.
display(pd.DataFrame(input_tables['OceanReservoirs']).set_index('Box ID'))
display(pd.DataFrame(input_tables['Atmosphere']).set_index('Box ID'))
baseline_parameters = pd.DataFrame(input_tables['ProcessParameters']).set_index('Parameter')
display(baseline_parameters.loc[list(BASELINE_PARAMETERS)])
```

	Description	Area (m2)	Volume (m3)	Temperature (degC)	Salinity	Pressure (bar)	Initial DIC (umol/kg)	Initial TA (umol/kg)
Box ID								
H_b	High-latitude surface	50000000000000	1760000000000000	2.0	35	17.6	2153	2345
L_b	Low-latitude surface	285000000000000	2850000000000000	21.5	35	5.0	1952	2288
D_b	Deep ocean	336000000000000	12900000000000000	2.0	35	240.0	2291	2399

	Total air (mol)	Initial CO2 (ppm)	Role
Box ID			
CO2_At	17786000000000000000	280	Prognostic CO2

	Value	Unit	Role	Description
Parameter				
thc	25.0	Sverdrup	Prescribed transport	Water transport on every leg of the overturning loop
mixing	30.0	Sverdrup	Prescribed transport	Equal exchange in both directions between high latitude and depth
poc_export	200.0	Tmol/yr	Prescribed export	POC carbon export from low-latitude surface to depth
rain_ratio	0.3	1	Prescribed ratio	PIC/POC carbon-export ratio at unit pump strengths
weathering_dic	12.0	Tmol/yr	Prescribed boundary flux	Carbonate weathering: one DIC and two TA equivalents per mole
alpha	0.6	1	Benchmark coefficient	Carbonate compensation coefficient
z0	-200.0	m	Benchmark elevation	Carbonate compensation reference elevation
piston_velocity	4.8	m/d	Prescribed exchange rate	Common gas transfer velocity at both surface boxes
opt_k_carbonic	13.0	1	Chemistry choice	PyCO2SYS carbonic-acid constant option
opt_pH_scale	3.0	1	Chemistry choice	PyCO2SYS pH scale option

A3. Exercise 03.1: write the paired fluxes and label the diagram

Step 1 — Write the paired fluxes

Question — equations on arrows

Complete both flux columns, including zero effects. Use one family expression for repeated water/gas arrows. Beside each row, note which changing model values affect its rate; write **fixed** for a prescribed constant rate. For net burial, identify the two fluxes being compared.

Use the notation and supplied hints below. Fill the table here or in the [student Excel worksheet](#), whose amber fields are answer spaces. Write symbolic equations, not Excel calculation formulas.

Notation

Use **amount fluxes** $J_a^{DIC}(t)$ [mol C/yr] and $J_a^{TA}(t)$ [equivalents/yr] for process a . Positive follows the arrow: subtract at its source and add at its destination. The DIC superscript tracks carbon removed from/returned to dissolved inventories, even when carried as CO₂ or particles. Gas exchange and net burial can be negative.

Symbol	Meaning / units
$DIC_i(t), TA_i(t)$	Evolving box concentrations [mol C/kg, eq/kg]; $i = L_b, H_b, D_b$
Q_{ij}	Water volume transport from i to j [m ³ /yr]; $\rho_i Q_{ij}$ converts it to mass transport [kg/yr]
ρ_i, m_i	ESBMTK density $\rho(T_i, S_i, p_i)$ [kg/m ³] and fixed box water mass $m_i = \rho_i V_i$ [kg]
P_0, r, W_0	Prescribed POC carbon export [mol C/yr], PIC/POC rain ratio [dimensionless], weathering carbon input [mol C/yr]
$E(t), D(t), B_{net}(t)$	Carbonate export, dissolution and signed net burial [mol C/yr]

Here p_i is water pressure; $pCO_{2,atm}$ is atmospheric CO₂ partial pressure. **Transport convention:** use physical ρQ for these equations; B2 explains the historical unit conversion retained in the benchmark code.

Supplied hints and flux table

For gas exchange, chemistry supplies aqueous CO₂:

$$c_i(t) = \mathcal{C}(DIC_i(t), TA_i(t); T_i, S_i, p_i).$$

Use area A_i [m²] and velocity v_i [m/yr]. Invasion uses $K_{0,i} pCO_{2,atm}(t)$; outgassing uses $c_i(t)$, both in mol/m³. Local solubility $K_{0,i}$ converts partial pressure to concentration. Form the two directional transfers before taking their difference. Supplied chemistry handles mol/kg and dry-air mole-fraction conversions.

For dissolution, **use this supplied dependency function:**

$$D(t) = \mathcal{D}(E(t), DIC_d(t), TA_d(t), z_{snow}(t); T_d, S_d, p_d, \alpha, z_0).$$

Here d means D_b, z_{snow} is the snowline, and α, z_0 are workbook sediment parameters. The function lists the inputs used to calculate dissolution; its equations and code are supplied.

In this model, POC export changes DIC but leaves TA unchanged (the **DIC-only closure** in the table).

Processes	Arrow	$J^{DIC}(t)$ [mol C/yr]	$J^{TA}(t)$ [eq/yr]	Inputs / hint
Q^*	Water: i -> j	Your expression	Your expression	Q_{ij} : water volume transport (m ³ /yr), from thc / mixing. Use source density and concentration.
G_L / G_H	Gas: atm -> surface i (signed)	Your expression	Your expression	piston_velocity, area; K_0 and c_i from local chemistry. Invasion minus outgassing.
POC	Organic export: your arrow	Your expression	Your expression	P_0 = poc_export; fixed, fully remineralized at depth; DIC-only closure.
PIC	Carbonate export: L_b -> module	Your expression	Your expression	r = rain_ratio (PIC/POC); CaCO ₃ formation removes 1 carbon and 2 TA equivalents.
W	Weathering: outside -> L_b	Your expression	Your expression	W_0 = weathering_dic; the model adds 2 TA equivalents per mole of DIC.
D	Dissolution: module -> D_b	$D(t)$ (supplied)	Your expression	D = sediment_response(E , DIC_d , TA_d , z_{snow} ; T_d , S_d , p_d , α , z_0). Reverse CaCO ₃ formation.
B_net	Net burial: module -> outside (signed)	Your expression	Your expression	Compare carbonate rain and dissolution, including old sediment; residual, not an extra drain.

▼ Reference: biological TA effects and weathering

The POC assumption omits nutrient/redox effects on TA. Nitrate uptake can increase TA and ammonium uptake can decrease it; a richer model needs the associated nutrient bookkeeping.

The model adds **1 mole of DIC and 2 equivalents of TA** through weathering. Real riverine input need not have exactly this 1:2 ratio.

Step 2 — Label the diagram

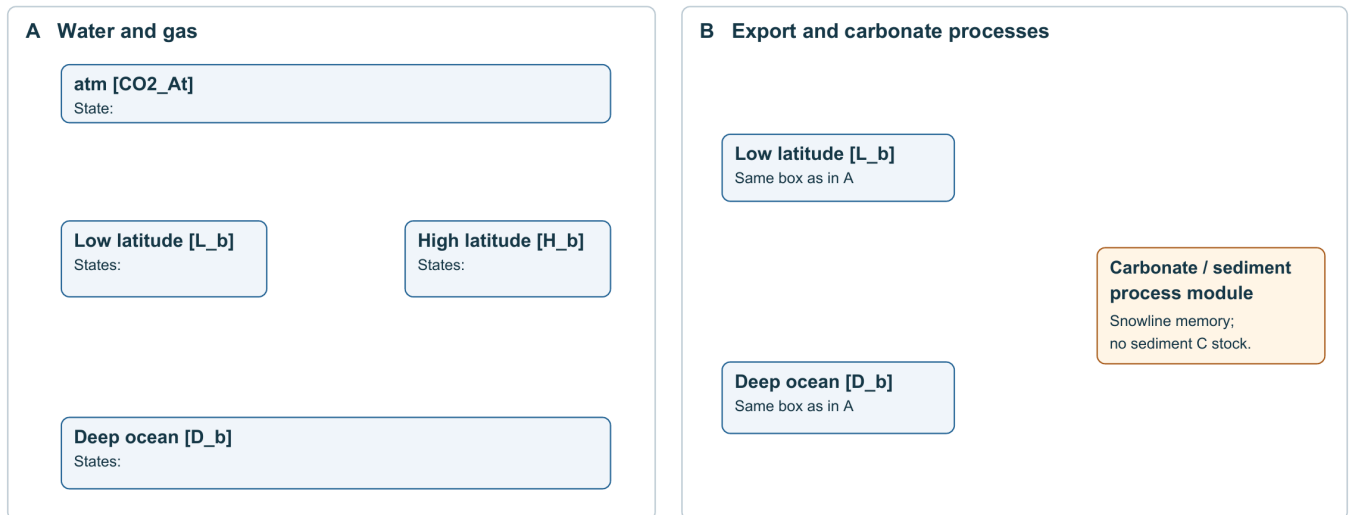
Question — label the diagram

Draw and label the arrows with these expressions. Add states and the active atm–ocn boundary; distinguish chemistry diagnostics and sediment memory. Check water balance at H_b.

Label water arrows with Q : circulation Q_{LH} , Q_{HD} , Q_{DL} and mixing $Q_{mix,down}$, $Q_{mix,up}$. The paired J equations describe the tracers they carry. F1–F8 remain paper aliases. Net burial is the signed amount left after dissolution, not an additional drain from deep water.

03.1 Label the model with your flux equations

Add states, arrows and the active atm + ocn boundary. Repeated boxes are the same reservoirs.



Q: water volume transport. J: paired DIC and TA amount fluxes from your table.

Include gas exchange, POC, PIC, weathering, dissolution and signed net burial. A paper sketch is sufficient.

Course adaptation of Wortmann et al. (2025), Fig. 3. Source corrections and input links are in notebook 03.

Step 3 — Check one internal transfer

For any internal dissolved transfer of $X = DIC$ or TA , **flux and concentration tendency differ**:

$$\left. \frac{dX_i}{dt} \right|_{i \rightarrow j} = -\frac{J_{ij}^X(t)}{m_i}, \quad \left. \frac{dX_j}{dt} \right|_{i \rightarrow j} = +\frac{J_{ij}^X(t)}{m_j}.$$

Question — check inventory cancellation

For one internal arrow, use the supplied tendency rule above to show equal-and-opposite inventory changes. Do not rewrite the effects in prose.

Instructor answer

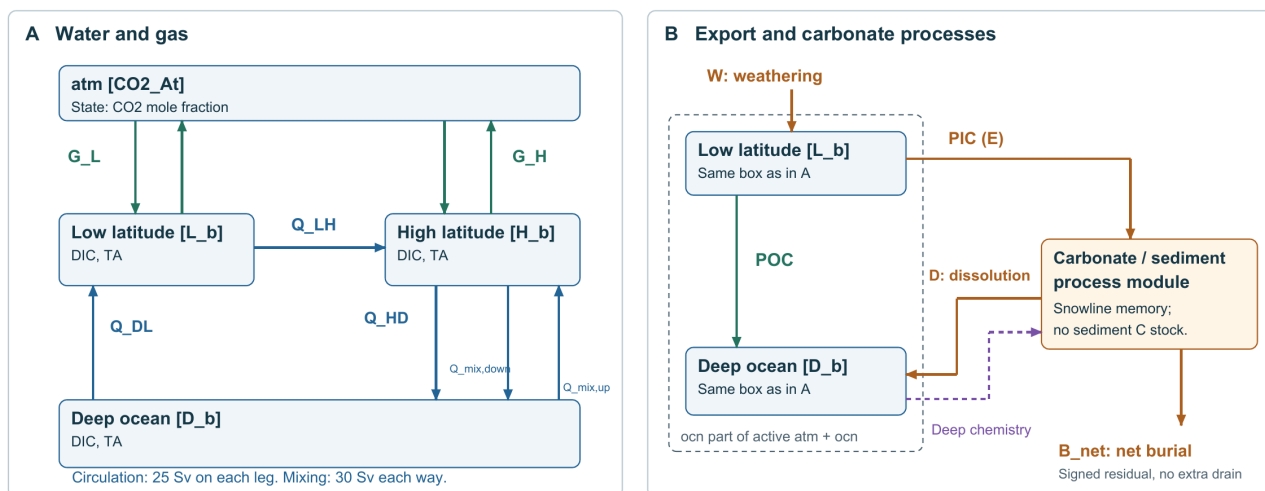
The completed equations are tabulated below. POC, PIC and weathering rates are fixed. Water-tracer fluxes scale with source concentration. Gas exchange uses atm CO₂ and aqueous CO₂; dissolution uses export, deep chemistry and snowline. Net burial is the difference between carbonate export and dissolution.

For any internal transfer, $m_i \dot{X}_i = -J^X$ and $m_j \dot{X}_j = +J^X$: their inventory sum is zero even when concentration changes differ. H_b gains and loses 25 Sv circulation plus 30 Sv mixing. DIC/TA and atm CO₂ are reservoir states; pH/aqueous CO₂ are chemistry diagnostics; snowline stores sediment history.

03 / 04 Model schematic

Repeated boxes are the same reservoirs. Use the companion table for paired DIC / TA equations.

INSTRUCTOR REFERENCE



Q: water volume transport [Sv]. 1 Sv = 10⁶ m³/s. J equations: companion flux table.

Solid arrows: material transfer. Dashed arrow: information. Grey outline: active ocean inventories.

Course adaptation of Wortmann et al. (2025), Fig. 3. Parameters and Excel-to-code links remain in notebook 03.

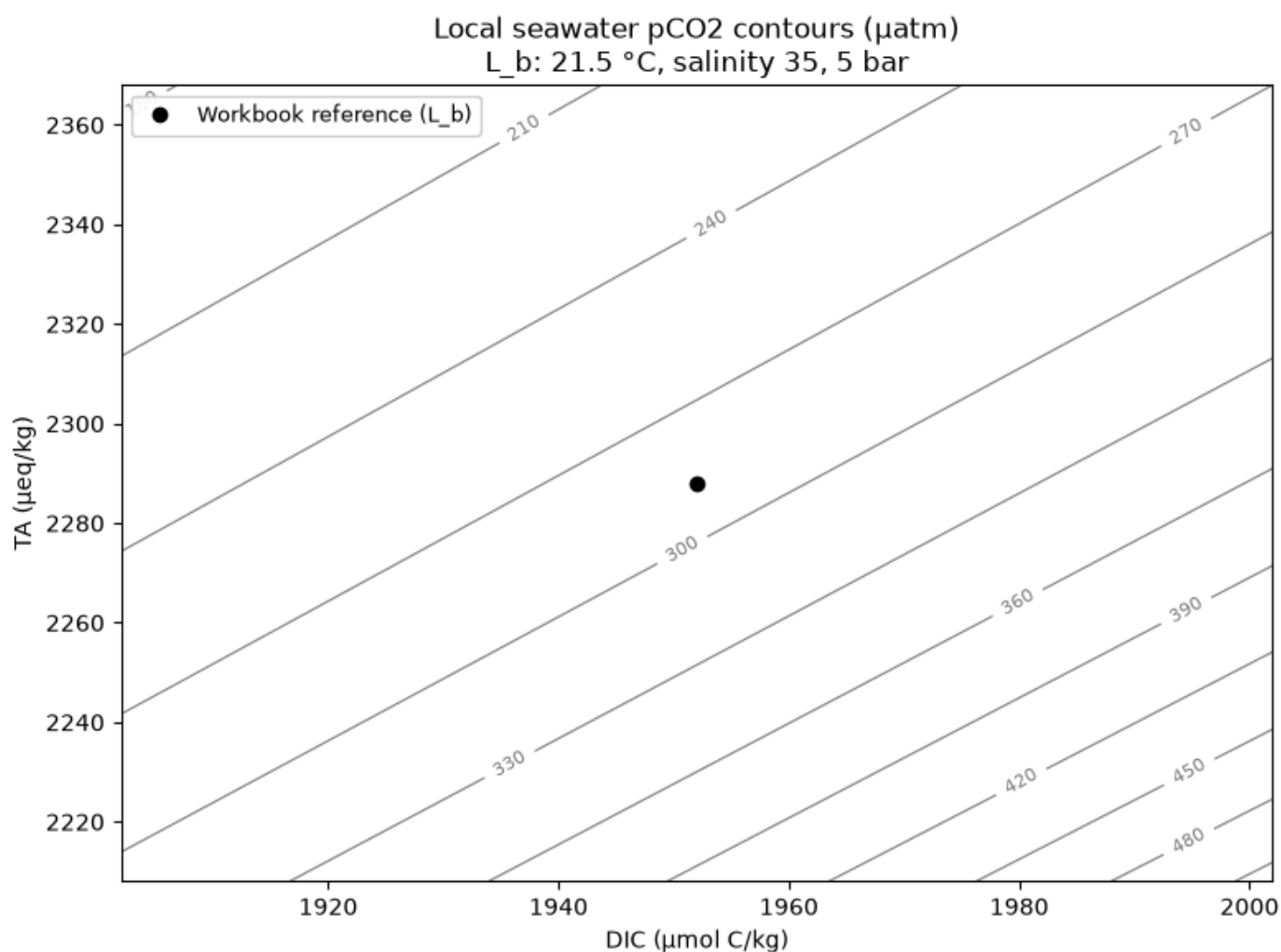
```
# Instructor reference: the student file omits this completed table.
from IPython.display import Markdown
from teaching_specification import flux_table_markdown
display(Markdown(flux_table_markdown()))
```

Process	Arrow	$J^{DIC}(t)$ [mol C/yr]	$J^{TA}(t)$ [eq/yr]	Inputs / hint
Q_*	Water: i -> j	$\rho_i Q_{ij} DIC_i(t)$	$\rho_i Q_{ij} TA_i(t)$	Q_ij: water volume transport (m3/yr), from thc / mixing. Use source density and concentration.
G_L / G_H	Gas: atm -> surface i (signed)	$v_i A_i [K_{0,i} pCO_{2,atm}(t) - c_i(t)]$	0	piston_velocity, area; K0_i and c_i from local chemistry. Invasion minus outgassing.
POC	Organic export: L_b -> D_b	P_0	0	P_0 = poc_export; fixed, fully remineralized at depth; DIC-only closure.
PIC	Carbonate export: L_b -> module	$E(t) = rP_0$	$2E(t)$	r = rain_ratio (PIC/POC); CaCO3 formation removes 1 carbon and 2 TA equivalents.
W	Weathering: outside -> L_b	W_0	$2W_0$	W_0 = weathering_dic; the model adds 2 TA equivalents per mole of DIC.
D	Dissolution: module -> D_b	$D(t)$ (supplied)	$2D(t)$	D = sediment_response(E, DIC_d, TA_d, z_snow; T_d, S_d, p_d, alpha, z0). Reverse CaCO3 formation.
B_net	Net burial: module -> outside (signed)	$B_{net}(t) = E(t) - D(t)$	$2B_{net}(t)$	Compare carbonate rain and dissolution, including old sediment; residual, not an extra drain.

Interpret your fluxes on the DIC–TA plane

Reuse 01's top view: DIC is horizontal, TA vertical, and each contour has constant seawater pCO₂ [μatm]. The supplied plot recalculates chemistry for the workbook's L_b inputs and benchmark T/S/p, rather than 01's uniform conditions. Its dot is an input reference, **not the saved starting state loaded in C3**. Treat each arrow as a local parcel change before transport and gas exchange.

```
# Understand and run: use the contours to interpret your paired fluxes.
from teaching_plots import plot_carbonate_process_plane
plot_carbonate_process_plane(P);
```



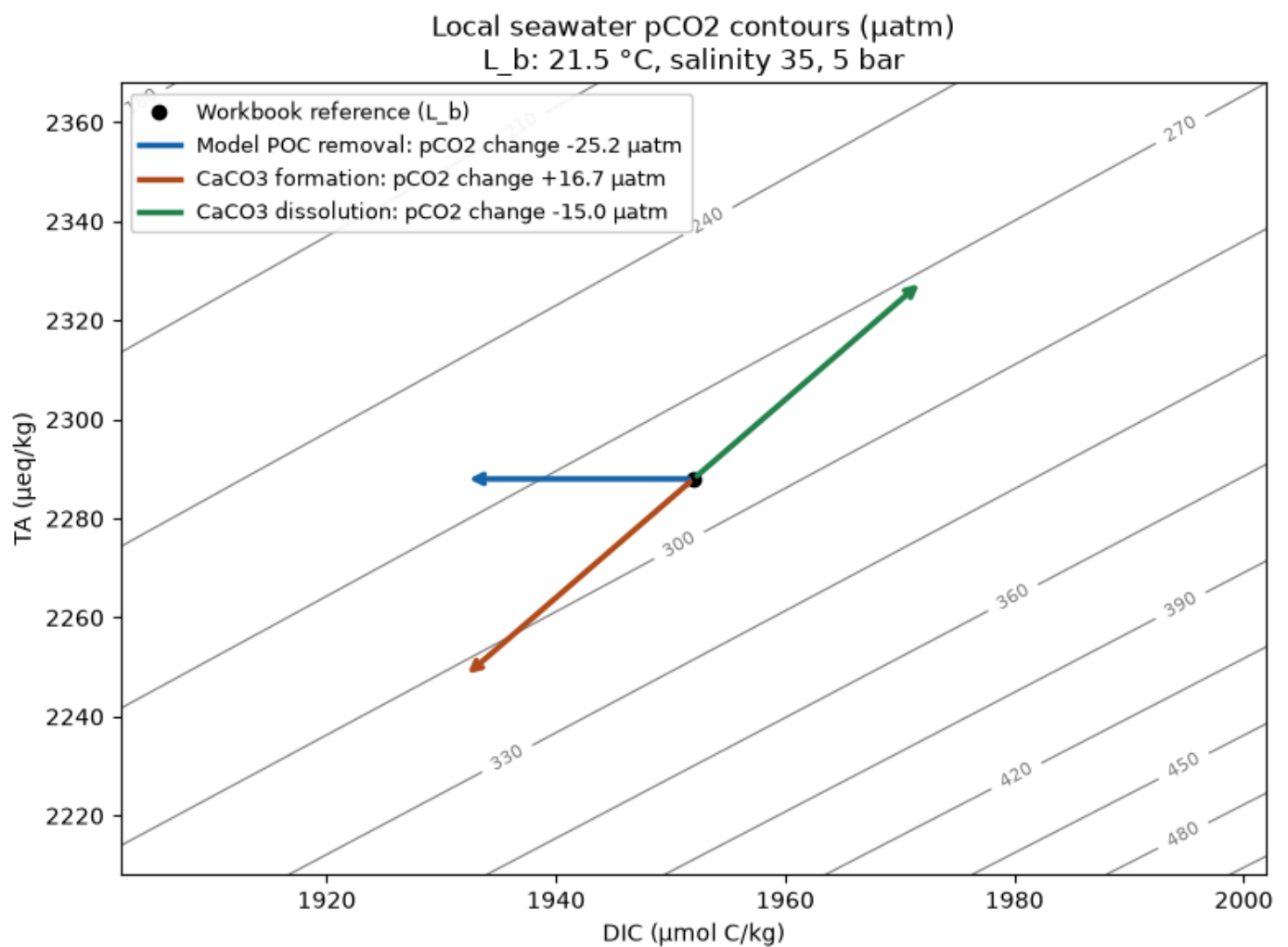
Question — interpret the paired fluxes

From the dot, remove 20 μmol C/kg through POC export and separately through CaCO₃ formation. Use your paired equations to draw the two DIC–TA vectors and predict the pCO₂ changes from the contours. Why do the signs differ? Reverse the carbonate vector for dissolution. Reuse these vectors in 03.4; no additional flux-effects paragraph is needed.

Instructor answer

Per mole C removed from the same parcel, model POC gives $(\Delta DIC, \Delta TA) = (-1, 0)$ and lowers pCO₂. CaCO₃ formation gives $(-1, -2)$ and raises pCO₂ near this reference: reduced TA leaves more of the remaining DIC as aqueous CO₂. Dissolution reverses this to $(+1, +2)$ and lowers pCO₂. TA changes are equivalents per mole C. These local vectors do not imply equal concentration changes between unequal box water masses.

```
# Supplied implementation – instructor process arrows
plot_carbonate_process_plane(P, show_processes=True);
```



A4. Reconcile your drawing with Excel

Compare the water and gas connection rows below with your arrows. Identify water routes by **source + sink + flux_id**: all three circulation rows use th_c , so that ID alone is not unique. Check boundary-node names and correct your diagram before construction. F_b declares a boundary node; it does not add a separate burial drain.

The loader validates names, units and water balance, then creates parameter records. You will create the reservoirs and connections in B1–B4.

▼ Optional reference: workbook-to-code links

Diagram content	Input owner / stable key	Code role
Geometry and states	OceanReservoirs / Box ID; Atmosphere	Reservoir specification
Water arrows	TransportConnections / source, sink, flux_id	Native bulk connections
Gas exchange	GasExchangeConnections / Atmosphere, Surface	Individual gas connections
POC and PIC	ProcessParameters / poc_export, rain_ratio	PIC derived from POC × ratio
Weathering	ProcessParameters / weathering_dic; BoundaryNodes / Fw	TA derived as twice weathering DIC
Dissolution / burial	Calculated responses, no prescribed flux row	Supplied carbonate module

The [flux worksheets](#) are teaching documentation, not live model inputs. POC/PIC/weathering topology and sediment coupling are supplied Python patterns. Changing baseline geometry, chemistry or rates requires another long equilibration run to prepare compatible starting values.

```
# Understand and run: compare these rows with the diagram you reconstructed.
display(pd.DataFrame(input_tables['TransportConnections']).sort_values('Order'))
display(pd.DataFrame(input_tables['GasExchangeConnections']).sort_values('Order'))
display(pd.DataFrame(input_tables['BoundaryNodes']).set_index('Box ID'))
# Optional lookup summarizes the same routes by unique connection ID.
from IPython.display import HTML
display(HTML(
    '<details><summary>Optional reference: connection cross-reference</summary>'
    + pd.DataFrame(workbook_connections(WORKBOOK)).set_index('ID').to_html()
    + '</details>'
))
```

	Order	source	sink	flux_id	Parameter	Species
0	1	H_b	D_b	mix_down	mixing	DIC, TA
1	2	D_b	H_b	mix_up	mixing	DIC, TA
2	3	L_b	H_b	thc	thc	DIC, TA
3	4	H_b	D_b	thc	thc	DIC, TA
4	5	D_b	L_b	thc	thc	DIC, TA

	Order	Atmosphere	Surface	Species	Parameter
0	1	CO2_At	H_b	CO2	piston_velocity
1	2	CO2_At	L_b	CO2	piston_velocity

	Type	Species	Role
Box ID			
Fw	Source	DIC, TA	External weathering supply
Fb	Sink	DIC, TA	Permanent burial boundary

▼ Optional reference: connection cross-reference

	Source	Sink	Species	Parameter
ID				
H_b_to_D_b@mix_down	H_b	D_b	DIC, TA	mixing
D_b_to_H_b@mix_up	D_b	H_b	DIC, TA	mixing
L_b_to_H_b@thc	L_b	H_b	DIC, TA	thc
H_b_to_D_b@thc	H_b	D_b	DIC, TA	thc
D_b_to_L_b@thc	D_b	L_b	DIC, TA	thc
CO2_At_to_H_b@gas	CO2_At	H_b	CO2	piston_velocity
CO2_At_to_L_b@gas	CO2_At	L_b	CO2	piston_velocity

B1. Exercise 03.2: map reservoirs

Model defines the clock, units and chemistry options; it is not a physical box or the system boundary. Run its supplied construction, then map the ocean records. Geometry remains explicit and ESBMTK calculates density from each box's T/S/P.

Question — map reservoir fields

Complete concentrations and geometry using the box record. Use species keys `M.DIC`, `M.TA` and geometry keys `area`, `volume`. The loop and T/S/P mapping are supplied. Trace one initial concentration from Excel to its state. Explain how volume and density convert ocn concentration to inventory, and why atm mole fraction needs a different conversion.

Local syntax: `box['dic']` retrieves a record field; `M.DIC` is the registered species, while `M.H_b.DIC` is its state in a particular reservoir. Dictionary keys are case-sensitive. The partial example below supplies the unchanged environmental fields.

```
# Supplied implementation: run this support code as provided.
# Supplied model clock and units.

M = Model(
    stop='20 yr',
    max_timestep='1 yr',
    element=['Carbon', 'Boron', 'Hydrogen', 'misc_variables'],
    mass_unit='mol',
    concentration_unit='mol/kg',
    opt_k_carbonic=P['opt_k_carbonic'],
    opt_pH_scale=P['opt_pH_scale'],
)

assert M.stop == Q('20 yr').to(M.t_unit).magnitude
assert M.DIC.name == 'DIC' and M.TA.name == 'TA'
```

ESBMTK 0.14.3.1.post0

Copyright (C) 2020 - 2026 Ulrich G.Wortmann

This program comes with ABSOLUTELY NO WARRANTY

This is free software, and you are welcome to redistribute it

under certain conditions; See the LICENSE file for details.

If you use ESBMTK for your research, please cite:

Wortmann et al. 2025, <https://doi.org/10.5194/gmd-18-1155-2025>

```
# Understand and run: partial record-to-constructor example.
high = P['boxes']['H_b']
high_specification = {'T': high['temperature'], 'P': high['pressure'], 'S': high['salinity']}
high_specification
```

```
{'T': 2.0, 'P': 17.6, 'S': 35.0}
```

```
# Choose and explain: complete marked choices; surrounding machinery is supplied.
def ocean_specification(box):
    # Exercise 03.2: complete the two mappings, using your diagram and the field hints.
    # BEGIN SOLUTION
    concentrations = {M.DIC: box['dic'], M.TA: box['ta']}
    geometry = {'area': box['area'], 'volume': box['volume']}
    # END SOLUTION
    return {'c': concentrations, 'g': geometry,
            'T': box['temperature'], 'P': box['pressure'], 'S': box['salinity']}

# Supplied repetition and boundary nodes.
box_parameters = {name: ocean_specification(box) for name, box in P['boxes'].items()}
for node in P['boundary_nodes']:
    box_parameters[node['name']] = {
        'ty': node['type'], 'sp': [getattr(M, species) for species in node['species']],
    }
species_list = initialize_reservoirs(M, box_parameters)
assert {M.L_b.name, M.H_b.name, M.D_b.name} == {'L_b', 'H_b', 'D_b'}
assert set(species_list) == {M.DIC, M.TA}
for box in (M.L_b, M.H_b, M.D_b):
    assert hasattr(box, 'DIC') and hasattr(box, 'TA')
```

```
# Understand and run: trace this supplied step and interpret its evidence.
display(pd.DataFrame(reservoir_inventory_rows(M)).set_index('Box'))
```

	Density (kg/m3)	Water mass (kg)	Initial carbon (mol)	Initial TA (mol eq)
Box				
H_b	1028.798771	1.810686e+19	3.898407e+16	4.246058e+16
L_b	1024.575722	2.920041e+19	5.699920e+16	6.681053e+16
D_b	1038.983451	1.340289e+21	3.070601e+18	3.215352e+18

Instructor answer

For a water mass $m = \rho V$, carbon inventory is $m DIC$ and TA inventory is $m TA$. Convert $\mu\text{mol/kg}$ to mol/kg first. Atmospheric carbon is $N_{atm} x_{CO_2}$ with dry-air mole fraction x ; ppm must be multiplied by 10^{-6} . `box['dic']` supplies an initial concentration, not a fixed target imposed during integration.

B2. Exercise 03.3: map water arrows and their law

For water volume transport Q_{ij} [m^3/yr], the physical tracer flux is

$$J_{ij}^X(t) = \rho_i Q_{ij} X_i(t), \quad X = \text{DIC or TA.}$$

The flux removes material at the source and adds the same amount at the destination.

Benchmark unit convention: O2 explicitly uses $Q\rho$. The historical benchmark instead passes volume scales through a native conversion to litres/yr while the states use mol/kg. We retain that nominal numerical conversion for reproduction; it is not a general dimensional recipe for new models. Inventories still use ESBMTK densities. Changing that conversion requires retesting the baseline and preparing compatible saved starting values.

As in O2, `Source_to_Sink@id` names a route. `ty` (or individual `ctype`) chooses the law; `sp` chooses species.

Law choice	Meaning	Supplied coefficient
<code>scale_with_concentration</code>	coefficient × current source concentration	<code>sc</code>
Fixed	prescribed amount/time; alias of <code>regular</code>	<code>ra</code>
<code>gasexchange</code>	invasion minus outgassing	gas-transfer settings

Question — implement your water arrows

Complete endpoints, transported species and `transport_type` from your reconciled specification. For the H_b-to-D_b mixing arrow, identify which concentration appears in its law. Would substituting destination concentration necessarily fail a whole-system inventory test?

```
# Choose and explain: complete marked choices; surrounding machinery is supplied.
connection_parameters = {}
for row in P['transport_connections']:
    # Exercise 03.3: map each row's source/sink and choose the carried species.
    # BEGIN SOLUTION
    source_name = row['source']
    sink_name = row['sink']
    transported_species = [M.DIC, M.TA]
    transport_type = 'scale_with_concentration'
    # END SOLUTION
    name = f"{source_name}_to_{sink_name}@{row['id']}"
    connection_parameters[name] = {
        'ty': transport_type, 'sc': P[row['parameter']],
        'sp': transported_species,
    }
create_bulk_connections(connection_parameters, M)
assert len(M.loc) == 2 * len(P['transport_connections'])
for row in P['transport_connections']:
    connections = [conn for conn in M.loc if conn.id == row['id']
                   and conn.source.parent.name == row['source']
                   and conn.sink.parent.name == row['sink']]
    assert len(connections) == 2
```

Instructor answer

The H_b-to-D_b flux uses H_b's current concentration. Using D_b's concentration would implement a different process, but equal source loss and destination gain could still conserve total inventory. Graph and equation checks therefore complement conservation.

B3. Exercise 03.4: map biological export

Use the POC and PIC arrows from your diagram. The PIC connections remove material from L_b without directly adding it to D_b: bp='sink' bypasses the named destination. The supplied sediment module returns only the dissolved fraction to D_b.

Question — choose pump species, laws and linked rates

Assign POC source/sink/species, select export_type from the law choices above, and link the PIC DIC and TA rates. Compare this export assumption with 02: if $DIC_s(t)$ changes while parameters stay fixed, what happens to export in each model?

```
# Choose and explain: complete marked choices; surrounding machinery is supplied.
# Supplied benchmark flux quantities.

M.tutorial_params = P
M.OM_export_reference = Q_(P['poc_export'])
M.CaCO3_export_reference = Q_(P['pic_export'])
M.OM_export = M.OM_export_reference
M.CaCO3_export = M.CaCO3_export_reference

# Exercise 03.4: select the POC arrow and link the two PIC rates.
# BEGIN SOLUTION
poc_source, poc_sink = 'L_b', 'D_b'
poc_species = M.DIC
export_type = 'Fixed'
pic_dic_rate = M.CaCO3_export
pic_ta_rate = 2 * pic_dic_rate
# END SOLUTION
create_bulk_connections(
    {
        f"{poc_source}_to_{poc_sink}@POM": {
            'sp': poc_species, 'ty': export_type, 'ra': M.OM_export,
        },
        'L_b_to_D_b@PIC_DIC': {
            'sp': M.DIC, 'ty': export_type, 'ra': pic_dic_rate, 'bp': 'sink',
        },
        'L_b_to_D_b@PIC_TA': {
            'sp': M.TA, 'ty': export_type, 'ra': pic_ta_rate, 'bp': 'sink',
        },
    },
    M,
)

M.OM_export_flux = M.flux_summary(filter_by='POM', return_list=True)[0]
M.CaCO3_export_flux = M.flux_summary(filter_by='PIC_DIC', return_list=True)[0]
M.rain_ratio = (
    M.CaCO3_export.to('Tmol/yr').magnitude
    / M.OM_export.to('Tmol/yr').magnitude
)

poc_connections = [c for c in M.loc if c.id == 'POM']
pic_dic_connections = [c for c in M.loc if c.id == 'PIC_DIC']
pic_ta_connections = [c for c in M.loc if c.id == 'PIC_TA']
assert len(poc_connections) == len(pic_dic_connections) == len(pic_ta_connections) == 1
assert pic_ta_connections[0].rate == 2 * pic_dic_connections[0].rate
assert M.rain_ratio == P['rain_ratio']
```

Instructor answer

Here POC and PIC export remain constant. In O2, $J_{pump}(t) = kDIC_s(t)$ changes proportionally with surface DIC despite constant k. The benchmark's fixed rain ratio links baseline PIC to POC; the computed ratio afterward is a consistency check, not an independent prediction.

Supplied dissolution and chemistry wiring

add_carbonate_system_2 implements your dissolution arrow using the actual PIC flux object, deep chemistry and the snowline. Run this supplied wiring; no manual dissolution connection is needed.

The **saturation horizon** marks calcite saturation = 1; the **compensation depth** is where the arriving carbonate rain dissolves completely. These diagnostics respond to current chemistry. The snowline can lag because old sediment takes time to dissolve; this can make dissolution exceed current rain and net burial negative.

The benchmark coefficient `alpha` was tuned in the ESBMTK reproduction; agreement with its reference is not independent evidence for that coefficient.

```
# Supplied implementation: run this support code as provided.
# Supplied chemistry and sediment coupling.

add_carbonate_system_1([M.L_b, M.H_b])
add_carbonate_system_2(
    r_sb=[M.L_b],
    r_db=[M.D_b],
    carbonate_export_fluxes=[M.CaCO3_export_flux],
    z0=P['z0'],
    alpha=P['alpha'],
)

assert hasattr(M.L_b, 'CO2aq') and hasattr(M.H_b, 'CO2aq')
assert M.D_b.cs2.function_input_data[0] is M.CaCO3_export_flux
```

B4. Exercise 03.5: connect the atmosphere

Map your A3 gas law to one connection per surface box. Each connection evaluates invasion minus outgassing, with equal-and-opposite atm and ocn transfers. Its endpoints define the positive direction; the net flux can reverse. Native code handles the conversions from dry-air mole fraction and mol/kg states.

Chemistry, gas exchange and circulation together produce solubility-related storage; no extra solubility-pump arrow is needed.

Question — map gas exchange

Assign `gas_source`, `gas_sink` and `gas_species` inside the supplied loop. `getattr(M, row['atmosphere'])` retrieves the named atmospheric object; `surface_box.DIC` selects a state within a surface box. Why does gas exchange update DIC although its law uses aqueous CO2?

```
# Choose and explain: complete marked choices; surrounding machinery is supplied.

# Supplied atmosphere and loop; complete the scientifically meaningful endpoints.

GasReservoir(
    name='CO2_At', species=M.CO2, species_ppm=P['pco2'],
    reservoir_mass=P['atmosphere_moles'],
)
```

```

for row in P['gas_exchange_connections']:
    surface_box = getattr(M, row['surface'])
    # Exercise 03.5: identify atmosphere, surface carbon state and gas species.
    # BEGIN SOLUTION
    gas_source = getattr(M, row['atmosphere'])
    gas_sink = surface_box.DIC
    gas_species = getattr(M, row['species'])
    # END SOLUTION
    Species2Species(
        source=gas_source,
        sink=gas_sink,
        species=gas_species,
        piston_velocity=P[row['parameter']],
        ctype='gasexchange',
        id=surface_box.name,
    )

gas_connections = [c for c in M.loc if c.ctype == 'gasexchange']
assert len(gas_connections) == 2
assert {c.sink for c in gas_connections} == {M.L_b.DIC, M.H_b.DIC}

```

Instructor answer

Aqueous CO₂ is a diagnostic fraction of DIC. Transfer of CO₂ therefore changes DIC, after which carbonate chemistry repartitions the dissolved carbon. Treating CO₂aq as an additional carbon stock would count carbon twice.

Supplied weathering connection

The workbook supplies `weathering_dic`; the loader sets the TA input to twice that rate. Run the supplied connection.

```

# Understand and run: trace this supplied step and interpret its evidence.
# Supplied external weathering arrow.

ConnectionProperties(
    source=M.Fw,
    sink=M.L_b,
    rate={M.DIC: P['weathering_dic'], M.TA: P['weathering_ta']},
    species=[M.DIC, M.TA],
    ctype='fixed',
    id='weathering',
)

weathering = [c for c in M.loc if c.id == 'weathering']
assert len(weathering) == 2
# Supplied metadata for the same boundary audit used in 04.
M.weathering_strength = 1.0

```

C1. Check the connections before running the model

The **connection graph** is the collection of boxes and connections you built. The supplied graph check matches endpoints, species, laws and process rates with the specification. Inspect this single summary and trace a water arrow, POC, linked PIC pair and gas connection to your diagram. A passing inventory check alone cannot establish that these are the intended equations.

```

# Understand and run: inspect one complete native graph summary.
connection_table = pd.DataFrame(audit_benchmark_graph(M))
display(connection_table)

```

	ID	Source	Sink	Law
0	H_b	CO2_At	H_b.DIC	gasexchange
1	L_b	CO2_At	L_b.DIC	gasexchange
2	PIC_DIC	L_b.DIC	D_b.DIC	regular
3	PIC_TA	L_b.TA	D_b.TA	regular
4	POM	L_b.DIC	D_b.DIC	regular
5	mix_down	H_b.DIC	D_b.DIC	scale_with_concentration
6	mix_down	H_b.TA	D_b.TA	scale_with_concentration
7	mix_up	D_b.DIC	H_b.DIC	scale_with_concentration
8	mix_up	D_b.TA	H_b.TA	scale_with_concentration
9	thc	D_b.DIC	L_b.DIC	scale_with_concentration
10	thc	D_b.TA	L_b.TA	scale_with_concentration
11	thc	H_b.DIC	D_b.DIC	scale_with_concentration
12	thc	H_b.TA	D_b.TA	scale_with_concentration
13	thc	L_b.DIC	H_b.DIC	scale_with_concentration
14	thc	L_b.TA	H_b.TA	scale_with_concentration
15	weathering	Fw.DIC	L_b.DIC	regular
16	weathering	Fw.TA	L_b.TA	regular

C2. Account for carbon and TA crossing the boundary

Use the **active atm–ocn boundary** drawn in A3: it encloses the atm and the three dissolved ocn inventories. The carbonate/sediment module is outside this inventory sum. For fixed box water masses m_i and atmospheric air amount N_{atm} ,

$$C_{atm+ocn}(t) = N_{atm}x_{CO_2}(t) + \sum_i m_i DIC_i(t), \quad A_{ocn}(t) = \sum_i m_i TA_i(t).$$

$C_{atm+ocn}$ is carbon in mol C; A_{ocn} is the TA inventory in equivalents. Here x_{CO_2} is the dry-air mole fraction, not ppm. TA has no atmospheric inventory.

Reuse A3's symbols: weathering supplies W_0 mol C/yr and $2W_0$ equivalents/yr. Net burial $B_{net}(t)$ is carbonate export minus dissolution, in mol C/yr; positive net burial removes material from the active inventories. There is **no experimental CO2 or TA addition** in this run, but weathering continues.

Question — build the whole-system budget

1. When you add the box inventories, why do water transport, gas exchange and POC export/remineralization make no net contribution?
2. Write $dC_{atm+ocn}/dt$ and dA_{ocn}/dt as **inputs minus outputs**, using weathering W_0 and net burial $B_{net}(t)$. Use the paired DIC/TA effects from your flux table.

Instructor answer

1. Internal transfers cancel. Each water or gas transfer removes carbon from one included box and adds the same amount to another. Water also transfers TA internally; gas exchange carries no TA. The model's POC transfer removes surface DIC and returns all of it as deep DIC through remineralization, with no TA effect.

2. Balance weathering against net burial. Using A3's $B_{net}(t) = E(t) - D(t)$,

$$\frac{dC_{atm+ocn}}{dt} = W_0 - B_{net}(t),$$
$$\frac{dA_{ocn}}{dt} = 2W_0 - 2B_{net}(t).$$

The factor two follows the model's weathering and carbonate stoichiometry: two TA equivalents per mole C. The balance equations have units mol C/yr and equivalents/yr, respectively. Negative net burial represents a net return from sediment to the dissolved inventories.

The inventories are constant only when weathering and net burial balance. A passing budget check means that inventory changes match the boundary fluxes; it does not require those changes to be zero.

C3. Load the saved state and check the next 20 years

`M.read_state(...)` loads the supplied saved values as this run's initial values: this is the **restart** introduced in A1. They come from a long benchmark run continued until changes were small (a nearly stationary state). Loading them does not create or repair connections; the equations you built determine what happens next.

Run 20 years with the existing weathering and export rates and no experimental addition. The supplied check allows at most **0.01 $\mu\text{mol/kg}$ DIC, 0.01 $\mu\text{eq/kg}$ TA, 0.01 ppm atm CO₂ and 0.01 m snowline movement**, measured at every saved time relative to the first. These are absolute teaching tolerances over this short run, not a relaxation timescale or proof of long-term stability.

The budget audit separately checks that carbon and TA inventory changes match the integrated weathering and net-burial fluxes from C2.

▼ Technical note: numerical budget tolerance

The audit's relative tolerance (2×10^{-5} of the evolving stock, plus a small absolute allowance) includes numerical integration of diagnostic fluxes. It evaluates net burial with the solver's own law; plotting diagnostics use a slightly different carbonate evaluation.

```
# Understand and run: state loading, integration and all audits are supplied.
M.read_state(directory=str(STATE))
run_model(M)
postprocess_carbonate_horizons(M)
budget_report = audit_complete_model(M)
drift_report = pd.DataFrame(audit_restart_drift(M)).set_index('State')
assert (M.D_b.zsat.c <= M.D_b.zcc.c).all()
display(drift_report)
print('Graph and PIC coupling checked; carbon/TA budgets close; short restart drift is within the stated limits.')
print({key: value for key, value in budget_report.items() if 'error' in key})
```

status=0

message=The solver successfully reached the end of the integration interval.

Execution took 0.03 CPU seconds, wall time = 0.03 seconds

	Maximum drift	Tolerance	Unit
State			
atm CO2	0.000006	0.01	ppm
L_b DIC	0.000042	0.01	umol/kg
L_b TA	0.000061	0.01	ueq/kg
H_b DIC	0.000068	0.01	umol/kg
H_b TA	0.000092	0.01	ueq/kg
D_b DIC	0.000126	0.01	umol/kg
D_b TA	0.000251	0.01	ueq/kg
sediment snowline	0.000171	0.01	m

Graph and PIC coupling checked; carbon/TA budgets close; short restart drift is within the stated limits.
{'max carbon error / initial stock': 5.599486194994163e-10, 'max TA error / initial stock': 1.0824442604743633e-09}

Question — explain the evidence

Name one error that the graph check could detect despite passing conservation, and explain what little change over these 20 years establishes and what it cannot establish about the real ocean. In your results, identify one state, one chemistry diagnostic and one flux. Use this explanation rather than another final report.

Instructor answer

A conservative but misdirected internal arrow may pass a global budget; the graph comparison can detect it. Little change over 20 years supports consistency between the reconstructed equations and the saved starting values over that interval. It does not prove long-term stability or observational validity. DIC is a state, pH a chemistry diagnostic, and dissolution a calculated flux. The sediment snowline is another evolving state, not a carbonate-ion diagnostic.

```
from model import initialize_model

reference = initialize_model(P, stop='20 yr', max_timestep='1 yr')
reference.read_state(directory=str(STATE))
run_model(reference)
postprocess_carbonate_horizons(reference)

for student_value, reference_value in (
    (M.CO2_At.c[-1], reference.CO2_At.c[-1]),
    (M.L_b.DIC.c[-1], reference.L_b.DIC.c[-1]),
    (M.L_b.TA.c[-1], reference.L_b.TA.c[-1]),
    (M.D_b.DIC.c[-1], reference.D_b.DIC.c[-1]),
    (M.D_b.zcc.c[-1], reference.D_b.zcc.c[-1]),
):
    assert abs(student_value - reference_value) <= max(abs(reference_value), 1.0) * 1e-10

print('Construction verified: the student-built and reusable models agree.')
```

ESBMTK 0.14.3.1.post0
Copyright (C) 2020 - 2026 Ulrich G.Wortmann
This program comes with ABSOLUTELY NO WARRANTY
This is free software, and you are welcome to redistribute it

under certain conditions; See the LICENSE file for details.

If you use ESBMTK for your research, please cite:

Wortmann et al. 2025, <https://doi.org/10.5194/gmd-18-1155-2025>

```
INFO:root:bypassing M.Conn_L_b_to_D_b_DIC_PIC_DIC._FPIC_DIC in M.D_b.DIC
INFO:root:bypassing M.Conn_L_b_to_D_b_TA_PIC_TA._FPIC_TA in M.D_b.TA
INFO:root:creating M_L_b_cs_generic_function
INFO:root:creating M_H_b_cs_generic_function
INFO:root:creating M_D_b_cs2_generic_function
INFO:root:creating M_H_b_DIC_gas_exchangeH_b_generic_function
INFO:root:creating M_L_b_DIC_gas_exchangeL_b_generic_function
INFO:root:=====
INFO:root:Intergration started at 2026-09-24 14:27:41.670305
INFO:root:r = M.H_b.DIC, r.c[0] = 2.15e-03, rtol = 2.15e-08
INFO:root:r = M.H_b.TA, r.c[0] = 2.35e-03, rtol = 2.35e-08
INFO:root:r = M.L_b.DIC, r.c[0] = 1.94e-03, rtol = 1.94e-08
INFO:root:r = M.L_b.TA, r.c[0] = 2.28e-03, rtol = 2.28e-08
INFO:root:r = M.D_b.DIC, r.c[0] = 2.30e-03, rtol = 2.30e-08
INFO:root:r = M.D_b.TA, r.c[0] = 2.40e-03, rtol = 2.40e-08
INFO:root:r = M.L_b.Hplus, r.c[0] = 5.40e-09, rtol = 5.40e-14
INFO:root:r = M.L_b.CO2aq, r.c[0] = 8.64e-06, rtol = 8.64e-11
INFO:root:r = M.H_b.Hplus, r.c[0] = 5.87e-09, rtol = 5.87e-14
INFO:root:r = M.H_b.CO2aq, r.c[0] = 1.57e-05, rtol = 1.57e-10
INFO:root:r = M.D_b.Hplus, r.c[0] = 1.22e-08, rtol = 1.22e-13
INFO:root:r = M.D_b.zsnow, r.c[0] = 4.81e+03, rtol = 4.81e-02
INFO:root:r = M.CO2_At, r.c[0] = 2.70e-04, rtol = 2.70e-09
INFO:root:Intergration finished:
  Number of evaluations of the right-hand side = 73
  Number of evaluations of the Jacobian = 1
  Number of LU decompositions = 19
  status=0
  message=The solver successfully reached the end of the integration interval.

INFO:root:

ipykernel_launcher.py stopped at 2026-09-24 14:27:41.711075
INFO:root:This run used 0.20 GB of memory

INFO:root:=====

status=0
message=The solver successfully reached the end of the integration interval.
```

Execution took 0.03 CPU seconds, wall time = 0.04 seconds

Construction verified: the student-built and reusable models agree.

You have completed 03. Keep the reconstructed diagram, flux table, four mappings and budget explanation. Reuse the diagram in 04.

Appendix: optional sediment reference

Outside the four-hour core. The executable module is supplied in notebook 03.

9.1 What the three depths mean

All depths are reported as positive metres below sea level.

Quantity	Meaning	Type in this model
z_{sat}	Calcite saturation horizon: depth where the water column changes from supersaturated to undersaturated, $\Omega_{\text{calcite}} = 1$	algebraic diagnostic
z_{cc}	Carbonate compensation depth: idealized depth at which the arriving modern carbonate rain is completely dissolved	algebraic diagnostic
z_{snow}	Carbonate snowline: deepest boundary of the existing reactive CaCO_3 sediment inventory	prognostic state with memory

Thus z_{sat} and z_{cc} respond immediately to current deep-ocean chemistry and export. The snowline can lag because previously deposited carbonate must dissolve before the sediment boundary moves upward. At the preindustrial steady state, z_{snow} lies close to z_{cc} .

9.2 How ESBMTK calculates z_{sat} and z_{cc}

Carbonate system 2 first calculates deep carbonate ion from deep DIC, TA, and hydrogen ion,

$$[\text{CO}_3^{2-}] = \frac{[\text{DIC}]}{1 + [\text{H}^+]/K_2 + [\text{H}^+]^2/(K_1K_2)}.$$

Pressure makes calcite less soluble upward and more soluble downward. ESBMTK represents the saturation carbonate concentration with a depth lookup table and solves the saturation condition $[\text{Ca}^{2+}][\text{CO}_3^{2-}] = K_{\text{sp}}(z)$. The implemented solution is

$$z_{\text{sat}} = z_{\text{sat},0} \ln \left(\frac{[\text{Ca}^{2+}][\text{CO}_3^{2-}]}{K_{\text{sp},0}} \right),$$

clipped between the top of the sediment domain and the maximum ocean depth.

The compensation depth also depends on the total PIC rain F_{export} ,

$$z_{\text{cc}} = z_{\text{sat},0} \ln \left(\frac{F_{\text{export}}[\text{Ca}^{2+}]}{K_{\text{sp},0}A_Dk_c} + \frac{[\text{Ca}^{2+}][\text{CO}_3^{2-}]}{K_{\text{sp},0}} \right).$$

Here A_D is the hypsometric seafloor area between z_0 and z_{max} , and k_c is the calcite dissolution coefficient. Lower deep-ocean carbonate makes both horizons shallower; greater carbonate rain makes z_{cc} deeper because more material can survive dissolution.

9.3 From ocean hypsometry to dissolution and burial

The model uses one-metre lookup tables for seafloor area and the carbonate concentration required for saturation. It divides the seafloor into chemically distinct depth intervals:

z_0	z_{sat}	z_{cc}	z_{snow}	z_{max}
supersaturated	undersaturated	no modern rain	old CaCO_3	
background term	kinetic dissolution	survives	may dissolve	

The mean carbonate rain per unit depositional area is

$$B_{AD} = \frac{F_{\text{export}}}{A_D}.$$

ESBMTK sums four contributions: a prescribed background response above z_{sat} , undersaturation-driven dissolution between z_{sat} and z_{cc} , complete dissolution of modern rain below z_{cc} , and dissolution of pre-existing sediment when $z_{\text{snow}} > z_{\text{cc}}$. In compact form,

$$F_{\text{diss}} = B_{\text{NS}} + B_{\text{DS}} + B_{\text{CC}} + B_{\text{PDC}},$$

$$F_{\text{burial}} = F_{\text{export}} - F_{\text{diss}}.$$

The dissolved fraction returns to the deep ocean as one DIC plus two TA equivalents per mole of CaCO_3 ,

$$J_{\text{deep,DIC}} = F_{\text{diss}}, \quad J_{\text{deep,TA}} = 2F_{\text{diss}}.$$

The PIC connection was created with `bp='sink'`, so its nominal deep-ocean sink is bypassed. Carbonate system 2 adds only the dissolved fraction back to the water. The undissolved residual has therefore left the active ocean inventory and represents burial. There is no explicit sediment reservoir in carbonate system 2; a model requiring explicit sediment mass would need a different module.

9.4 Why z_{snow} is different

The snowline is initialized near 4750 m and is registered as an ESBMTK state variable. When the instantaneous compensation depth becomes deeper than the snowline, the snowline relaxes downward toward it. When the snowline is deeper than the compensation depth, previously deposited carbonate between them dissolves and the boundary shoals according to

$$\frac{dz_{\text{snow}}}{dt} = - \frac{B_{\text{PDC}}}{A(z_{\text{snow}})I_{\text{CaCO}_3}},$$

where $A(z_{\text{snow}})$ is seafloor area per depth interval and I_{CaCO_3} is the dissolvable sediment inventory per unit area. This prognostic boundary gives carbonate compensation its memory and long response time.

During integration, carbonate system 2 repeatedly calculates the instantaneous z_{sat} , z_{cc} , and dissolution flux needed by the ODE, but only z_{snow} is retained as a dynamic state. After integration, `postprocess_carbonate_horizons` reconstructs the saved z_{sat} , z_{cc} , F_{diss} , and F_{burial} series from the stored chemistry, PIC flux, and already-integrated snowline.